FH | JOANNEUM
System Test Engineering

Permittivity Meter ReDesign

Project Leader: Mlinar Johannes

Project Number: ECE.23.D

Version: 1.2

Date: 22.10.2025

Inhaltsverzeichnis

1 System Structure

1.1 Microcontroller Overview

The core of the system is the STM32L4xx series microcontroller. This ultra-low-power MCU was selected for its efficient peripheral set, specifically the advanced analog peripherals (ADC/DAC) required for signal processing and the flexible timer architecture needed for high-frequency signal generation.

1.2 System Architecture Diagram

The software is designed in a strict hierarchical layer structure to ensure modularity and ease of testing.

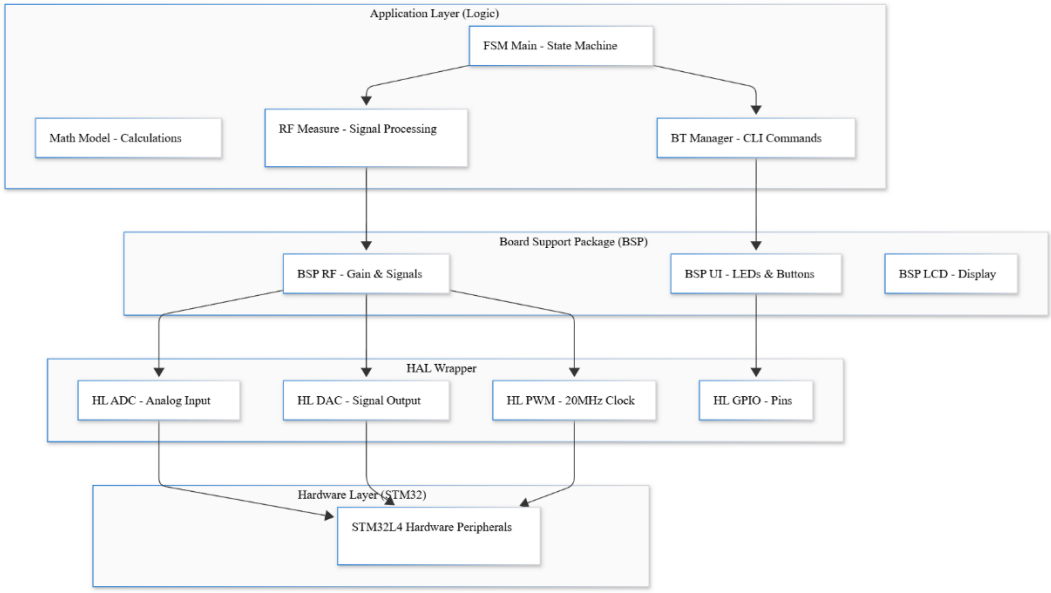


Figure 1:

2 Quartz stability testing

To ensure the accuracy of the undersampling measurement mechanism and the 20MHz square wave output, the stability of the 20 MHz excitation signal was analyzed. A comparative analysis was conducted to evaluate the phase jitter introduced by two different clock sources: the internal High-Speed Internal oscillator (HSI) versus the external High-Speed External crystal (HSE). The native frequency of the HSI oscillator on the STM32L4xx is 16 MHz. To permit a direct comparison with the 20 MHz External Crystal (HSE), the PLL was utilized to boost the HSI frequency to 20 MHz.

The PC8 pin was configured in Microcontroller Clock Output (MCO) mode to allow for external measurement of the signal.

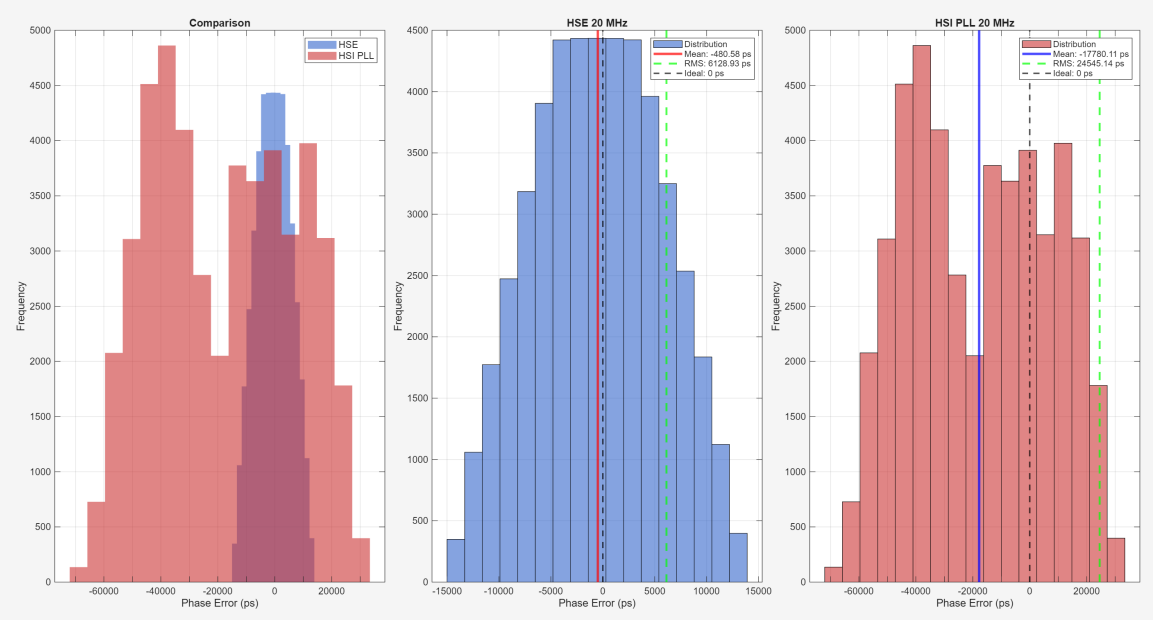


Figure 2: comparing the jitter performance of the External Crystal Oscillator (HSE) and the Internal RC Oscillator (HSI) with PLL to match the HSE 20MHz.

The external crystal (HSE) provides approximately 4x better stability (lower RMS jitter) compared to the internal oscillator. Consequently, the HSE is mandated as the system clock source. This reduction in jitter is critical for coherent undersampling, directly improving the Signal-to-Noise Ratio (SNR) of the permittivity calculations.

3 System Implementation

3.1 PWM Generation (Signal Source)

The system generates a stable 20 MHz square wave excitation signal via Pin PA0 (TIM2 Channel 1). The timer is driven by an 80 MHz clock with a prescaler of 0 and a period of 3 (Pulse 2), resulting in an effective 50% duty cycle at the target frequency.

//Explain the code

Code 1: Configure timer for desired frequency and duty cycle in hal_pwm.c

```
/* Core/Src/hl/hal_pwm.c */
static PWM_StatusTypeDef PWM_Configure_Timer(uint32_t frequency_hz, uint8_t
duty_cycle)

{
    // ... validation checks ...
    uint32_t prescaler = 0;
    uint32_t period = 0;
    // Try to find optimal prescaler and period values
    // PWM_TIMER_CLOCK is 80 MHz (SYSCLK)
    for (prescaler = 0; prescaler < 65536; prescaler++)
    {
        uint32_t timer_freq = PWM_TIMER_CLOCK / (prescaler + 1);
        period = (timer_freq / frequency_hz) - 1;
        // Break if we find a valid period that fits in the 16/32-bit
register
        if (period <= 65535 && period > 0)
        {
            break;
        }
    }
    // Apply new configuration
    htim_pwm->Init.Prescaler = prescaler;
    htim_pwm->Init.Period = period;
    // Update hardware registers directly
    __HAL_TIM_SET_PRESCALER(htim_pwm, prescaler);
    __HAL_TIM_SET_AUTORELOAD(htim_pwm, period);
    // ... duty cycle calculation ...
    return PWM_OK;
}
```

Code 2: main initialization in main.c

```
/* Core/Src/main.c */
/* Initialize PWM Driver (20 MHz excitation signal on PA0) */
  HAL_PWM_Init(&htim2);              // 1. Link driver to TIM2 handle
  HAL_PWM_SetFrequency(20000000UL); // 2. Public call -> triggers private
PWM_Configure_Timer()
  HAL_PWM_SetDutyCycle(50);          // 3. Set to 50%
  HAL_PWM_Start();                   // 4. Enable output
```

3.2 Analog-to-Digital Conversion

Signal acquisition is performed using *hal_adc.c*, implementing an undersampling technique to analyze high-frequency signals.

Precise undersampling of high-frequency signals is achieved by driving the 12-bit ADC1 (channel 1) with a hardware-timed 122.5 kHz trigger from TIM6, ensuring the phase coherence and zero-jitter performance necessary for accurate measurement via DMA double buffering.

A significant design challenge was enforcing strict timing control. The default initialization generated by STM32CubeMX sets the ADC to "Software Trigger" and "Continuous Mode," which causes sampling jitter.

To resolve this, the HL_ADC_Init function explicitly overrides the CubeMX configuration at runtime to force hardware timer triggering.

Code 3:

```
/* Core/Src/hl/hal_adc.c */
ADC_StatusTypeDef HL_ADC_Init(ADC_HandleTypeDef *hadc, TIM_HandleTypeDef
*htim)
{
    // 1. Configure the Trigger Timer (TIM6)
    // Target Frequency: 122.5 kHz (Derived from 80 MHz System Clock)
    htim_trigger->Init.Prescaler = 0;
    htim_trigger->Init.Period = 652; // (80,000,000 / 122,500) - 1

    // ... Timer initialization ...
    // 2. CRITICAL: Override CubeMX default ADC configuration
    // We disable continuous mode to ensure the ADC waits for the Timer
Trigger
    // before EVERY conversion. This locks the sampling measure to the clock.
    hadc_local->Init.ContinuousConvMode = DISABLE;
    hadc_local->Init.ExternalTrigConv = ADC_EXTERNALTRIG_T6_TRGO;
    hadc_local->Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_RISING;
    if (HAL_ADC_Init(hadc_local) != HAL_OK)
    {
        return ADC_ERROR;
    }
    return ADC_OK;
}
```

3.2.1 DMA Data Management

The system uses a "Ping-Pong" buffering strategy. The DMA buffer size is double the processing length (2 * ADC_BUFFER_SIZE). Frame completion callbacks update a pointer to the valid half of the buffer, allowing the application to process one half while the ADC blindly fills the other, ensuring zero dead time.

This mechanism allows the ADC to write conversion results directly into a RAM array (dma_buffer) in the background, completely bypassing the CPU. This ensures that no samples are missed due to CPU processing delays or interrupt latency.

Code 4:

```
void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc)
{
    // First half ready (0 to N-1)
    ready_buffer_ptr = &dma_buffer[0];
    buffer_ready_flag = true;
}
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
    // Second half ready (N to 2N-1)

    //Sampling-time test
    /HAL_GPIO_TogglePin(MEAS_LED_GPIO_Port, MEAS_LED_Pin);

    ready_buffer_ptr = &dma_buffer[ADC_BUFFER_SIZE];
    buffer_ready_flag = true;
}
```

To validate the real-time performance of the ADC subsystem, an oscilloscope measurement was performed. A test signal was toggled within the HAL_ADC_ConvCpltCallback function, which executes every time the ADC completes a full DMA buffer cycle with Buffer size *N*.

$$N = 1024 \text{ samples } (configured\ as\ a\ Double\ Buffer\ of\ 512 \times 2)$$

$$T_{int} = \frac{N}{f_s} = \frac{1024}{122.5 \text{ kHz}} \approx 0.00835918 \text{ s} \approx 8.359 \text{ ms}$$

$$T_{period} = 2 \times T_{int} = 2 \times 8.359 \text{ ms} \approx 16.718\ ms$$

$$f_{calc} = \frac{1}{T_{period}} = \frac{1}{0.016718 \text{ s}} \approx 59.81\ Hz$$

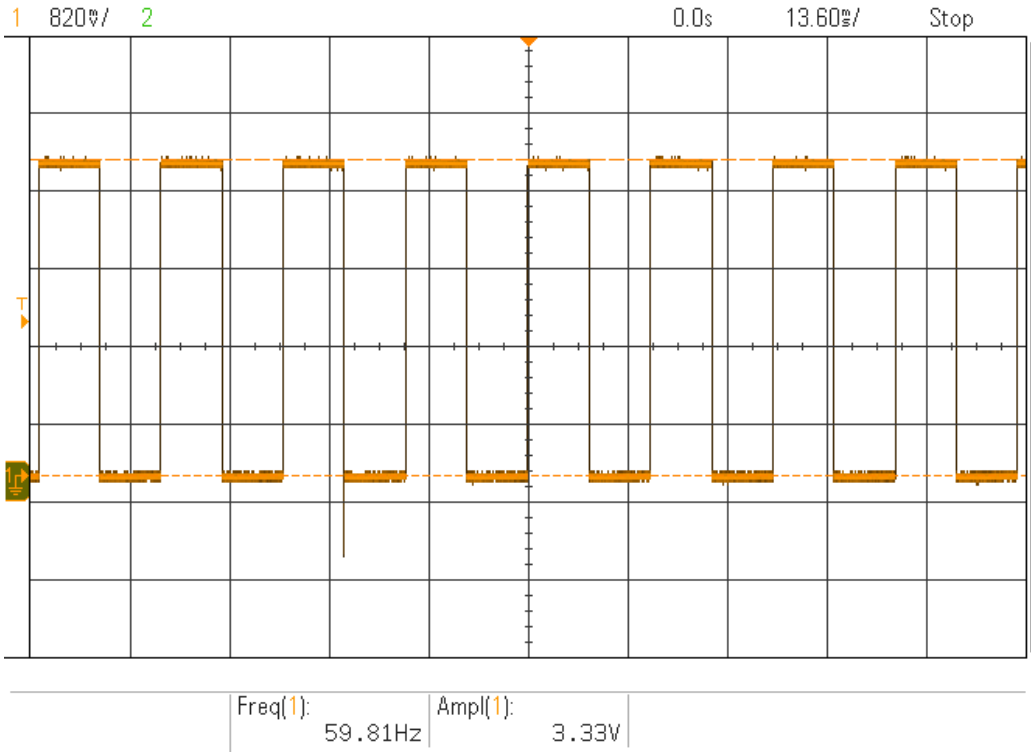


Figure 3:

To measure the frequency of the ADC sampling, its pin (PC0) was left floating which would fill the two DMA buffers. A complete square wave cycle requires two interrupts (one to go High, one to go Low). The measured frequency matches the theoretical calculation exactly. This confirms that the ADC is sampling precisely at 122.5 kHz with zero drift, validating that the hardware timer (TIM6) is correctly driving the acquisition timing.

3.2.2 Mathematical Derivation of the Alias Frequency

The observed frequency of ~30.6 kHz is not random; it is the precise mathematical result of undersampling. The alias frequency (f_a) is calculated by finding the absolute difference between the signal frequency $(f_{sig} = 20MHz)$ and the nearest integer multiple N of the sampling rate $(f_s = 122.5\,kHz)$

$$f_a = |f_{sig} - (N \times f_s)|$$

$$f_a = |20{,}000{,}000 - (163 \times 122{,}511)|$$

$$f_a = |20{,}000{,}000 - 19{,}969{,}293|$$

$$f_a \approx 30{,}707 \text{ Hz}$$

3.2.3 Aliasing Sensitivity and ADC Aperture

To validate the system's core capability, a physical closed-loop test environment was established. The microcontroller's PWM Output Pin (PA0) was physically connected directly to the ADC Input Pin (PC0) using a short jumper wire (~2cm). This configuration created a loopback system where the MCU generated a 20 MHz square wave and immediately attempted to read it back, isolating the signal path from external environmental variables as much as possible.

Moreover, Reducing the sampling time to the hardware minimum of 2.5 cycles eliminated signal averaging, validating 20 MHz undersampling and mandating the strict use of ADC_SAMPLETIME_2CYCLES_5.

Data Capture Strategy (UART Interception)

The main loop of the firmware was reprogrammed to dump the raw contents of the ADC memory buffer to the UART Serial Port. A PC-side Serial Terminal was employed to capture this raw text stream, providing access to the exact numerical values (0-4095) registered by the ADC.

```
// Temporary: Dump one buffer of data to UART for Excel analysis
  // 1. Wait for buffer
  while (!HL_ADC_IsBufferReady()) { HAL_Delay(10); }

  uint16_t *buf = HL_ADC_GetBuffer();

  // CRITICAL: Stop ADC immediately so it doesn't overwrite data while
we print!
  HAL_ADC_Stop_DMA(&hadc1);

  char msg[32];
  sprintf(msg, "START_DUMP\r\n");
  HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 100);

  // dump the whole 512 samples
  for (int i = 0; i < 512; i++)
  {
      sprintf(msg, "%d\r\n", buf[i]);
      HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 10);
  }

  sprintf(msg, "END_DUMP\r\n");
  HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 100);
```

Results

Upon rectifying the aperture setting (shutter speed) and applying the Freeze-Frame capture method, the system revealed a perfect rail-to-rail signal aliased to 30.6 kHz. The captured data exhibited a clear High-Low-Low-High pattern (cycle), definitively proving that the hardware bandwidth and sampling methodology are capable of detecting the 20 MHz wave with high fidelity.

$$f_{pattern} = \frac{f_s}{4} = \frac{122{,}511\ Hz}{4} \approx 30{,}627\ \text{Hz}$$

Some of the DMA memory dump output

```
START_DUMP
3701
0
59
4024
3701
0
73
4024
.
.
.

3708
3
65
4026
3717
0
58
4024
END_DUMP
```

3.3 DAC and Signal Control

The Digital-to-Analog Converter (DAC) acts as the bridge between the digital control logic and the analog measurement circuit. It drives the varactor diodes (varicaps) to tune the sensor's resonant frequency and adjusts the circuit quality factor (Q-factor).

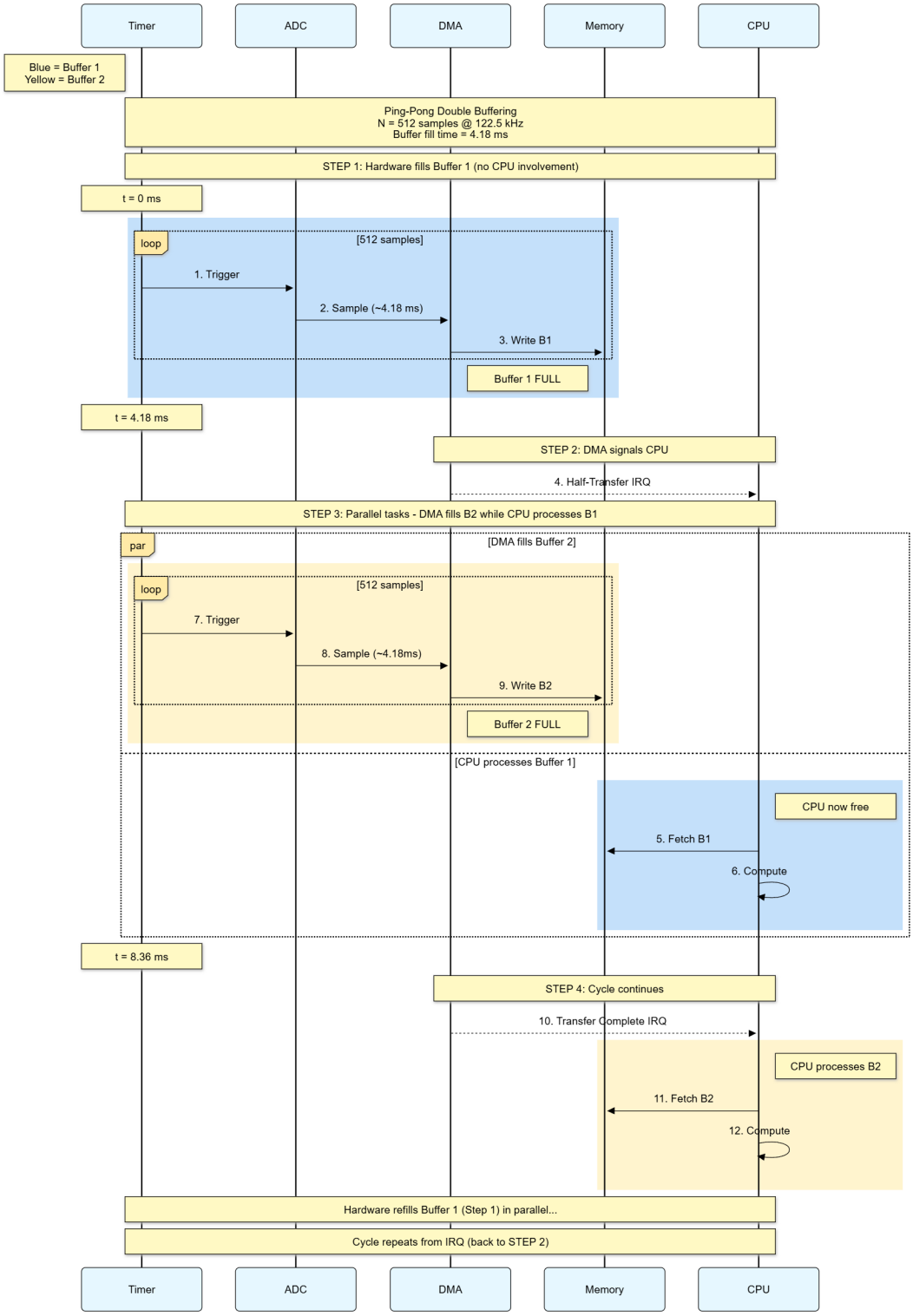
Table 1: DAC1 Configuration and Channel Mapping

Feature	Parameter	Value / Description	Code / Macro Reference
Hardware Instance	Instance	DAC1	hdac1 (in main.c)
	Resolution	12-bit	*(Standard STM32 L4 DAC)*
Electrical Limits	Voltage Range	0 V – 3.3 V	DAC_VOLTAGE_REF
Digital Limits	Value Range	0 – 4095	DAC_MAX_VALUE
Channel 1	GPIO Pin	PA4	FRQ_TN_Pin
	Function	Frequency Tuning Voltage	DAC_CH_FREQ_TUNE
Channel 2	GPIO Pin	PA5	Q_FACT_TN_Pin
	Function	Q-Factor Control Voltage	DAC_CH_Q_FACTOR

Code 5:

```
/* Core/Src/hl/hal_dac.c */
/**
 * @brief Set DAC output voltage for a specific channel
 * @param channel: Target Channel (FREQ_TUNE or Q_FACTOR)
 * @param voltage: Desired voltage (0.0V - 3.3V)
 */
DAC_StatusTypeDef HL_DAC_SetVoltage(DAC_ChannelTypeDef channel, float
voltage)
{
    // 1. Safety Check
    if (voltage < 0.0f || voltage > DAC_VOLTAGE_REF)
    {
        return DAC_ERROR_INVALID_PARAM;
    }

    uint32_t raw_value = (uint32_t)((voltage / DAC_VOLTAGE_REF) *
DAC_MAX_VALUE);
    // 3. Hardware Write
    if (HAL_DAC_SetValue(hdac_local, channel, DAC_ALIGN_12B_R, raw_value) !=
HAL_OK)
    {
        return DAC_ERROR;
    }
    return DAC_OK;
}
```


Timer
ADC
DMA
Memory
CPU
Blue = Buffer 1
Yellow = Buffer 2
Ping-Pong Double Buffering
N = 512 samples @ 122.5 kHz
Buffer fill time = 4.18 ms
STEP 1: Hardware fills Buffer 1 (no CPU involvement)
t = 0 ms
loop
[512 samples]
1. Trigger
2. Sample (~4.18 ms)
3. Write B1
Buffer 1 FULL
t = 4.18 ms
STEP 2: DMA signals CPU
4. Half-Transfer IRQ
STEP 3: Parallel tasks - DMA fills B2 while CPU processes B1
par
[DMA fills Buffer 2]
loop
[512 samples]
7. Trigger
8. Sample (~4.18ms)
9. Write B2
Buffer 2 FULL
[CPU processes Buffer 1]
CPU now free
5. Fetch B1
6. Compute
t = 8.36 ms
STEP 4: Cycle continues
10. Transfer Complete IRQ
CPU processes B2
11. Fetch B2
12. Compute
Hardware refills Buffer 1 (Step 1) in parallel...
Cycle repeats from IRQ (back to STEP 2)
Timer
ADC
DMA
Memory
CPU

3.4 Communication Interfaces

The system features two independent serial communication channels: one for wired PC debugging and another for wireless Bluetooth control.

Code 6: Communication Interfaces Configuration

Interface	Parameter	Value / Description	Code Reference
USB-Serial	Hardware Instance	USART2	-
(ST-LINK VCP)	Pins	TX: PA2 RX: PA3	(Standard Nucleo VCP)
	Settings	115200 baud, 8N1	-
	Flow Control	None	-
	Usage	Primary Debugging (CLI, printf)	-
Bluetooth	Hardware Instance	UART4	huart4
(u-blox NINA-B1)	Pins	TX: PC10 RX: PC11 RTS: PA15 CTS: PB7	NINA_TX_Pin (PC10)*
	Settings	115200 baud, 8N1	huart4.Init.BaudRate
	Flow Control	RTS/CTS Enabled	UART_HWCONTROL_RTS_CTS
	Usage	Wireless Data Logging & App Connectivity	-

4 Appendix